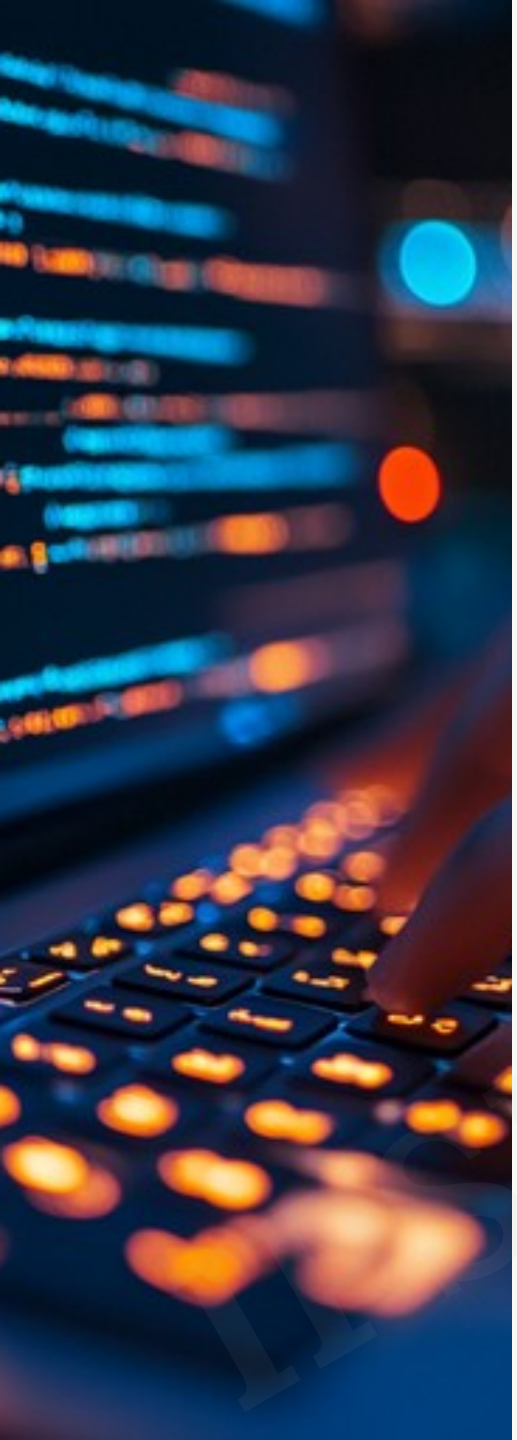


第 2 章、電腦的硬體結構

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

定址模式

定址模式

在CPU中，根據不同的指令操作需求，可以選擇不同的定址模式來存取記憶體中的資料。常見的定址模式包含：

- **立即載入法** (Immediate Addressing Mode)
- **相對定址法** (Relative Addressing Mode)
- **索引定址法** (Indexed Addressing Mode)
- **絕對定址法** (Absolute Addressing Mode)

IPSI Lab, Yuntech-CSIE

立即載入法

- 立即載入法將指令中的常數值直接載入到暫存器中。
- 非常簡單且快速的資料載入方式，通常用於設定暫存器初值或進行簡單的加法運算。
- **範例**：
  - `LDI R1, 100`：將數值 100 直接載入暫存器 R1 中。
  - `LDI R2, R1+50`：將 R1 中的值加上 50，結果載入 R2 中。

IPSI Lab, Yuntech-CSIE

優點與缺點

- **優點**：執行速度快，適用於常數值的設定與簡單計算，減少對記憶體的存取次數。
- **缺點**：僅能處理小範圍的數值，無法進行複雜運算或存取動態資料。
  - 在 RISC 架構中，立即值可能受到位元數的限制，因此無法載入過大數值。

IPSI Lab, Yuntech-CSIE

相對定址法

- 相對定址法通過一個基底暫存器與指令中的偏移量來存取記憶體中的資料。
- 這種方法允許指令動態計算存取位址，適用於操作相對位置的資料。
- **範例**：
  - `LD R1, [R2+100]`：從記憶體中取得 R2 位址加上 100 偏移量的資料，並存入 R1 中。
  - `ST R1, [R3+200]`：將 R1 中的資料存入記憶體 R3 加 200 的位址中。

IPSI Lab, Yuntech-CSIE

應用場景

- 相對定址常用於存取結構化資料、陣列或表格中的特定元素。
- 例如，當操作一個陣列時，可以使用基底暫存器記錄陣列的起始位置，然後利用偏移量來存取陣列中的不同元素。

優點與缺點

- **優點**：能動態計算位址，靈活存取記憶體中的資料。
- **缺點**：需要額外的位址計算，因此指令執行速度可能會略慢於立即載入法。

IPSI Lab, Yuntech-CSIE

索引定址法

- 索引定址法結合基底暫存器與索引暫存器的值來計算最終的記憶體位址，
- 這種方式非常適合操作陣列和多維資料結構。
- **範例**：
  - `LDR R1, [R2+R3]`：將基底暫存器 R2 與索引暫存器 R3 的和作為位址，載入對應位址的資料到 R1 中。
  - `STR R4, [R2+R5]`：將 R4 中的資料存入 R2 與 R5 的和所對應的記憶體位址中。

應用場景

- **陣列存取**：在處理陣列時，索引定址能有效地定位並存取多維陣列中的資料。基底暫存器可用來記錄陣列起始位址，索引暫存器則可用來計算偏移位置，特別是處理大資料集合時，這種方法能顯著提升效能。

範例- 陣列存取

假設有一個整數陣列 `a[100]`，存取其中某個元素：

- 基底位址存放於 `R2`，索引存放於 `R3`。為了取得第 `i` 個元素：
  - `LDR R1, [R2+R3*4]`
    - 從陣列的基底位址開始，根據索引計算偏移量，取得對應的陣列元素並載入到 R1 中。
    - `R3*4` 是因為每個整數佔據 4 個位元組 (bytes)，這樣可以精確定位每個元素在記憶體中的位址。

IPSI Lab, Yuntech-CSIE

相對定址法 vs. 索引定址法

相對定址:
適用於存取結構化資料、陣列或表格中的特定元素。
索引定址法:
適用於存取陣列和多維資料結構。

➡ 二者都用於存取結構化資料
二者的差別是什麼!?

IPSI Lab, Yuntech-CSIE

CPU0 的程式執行

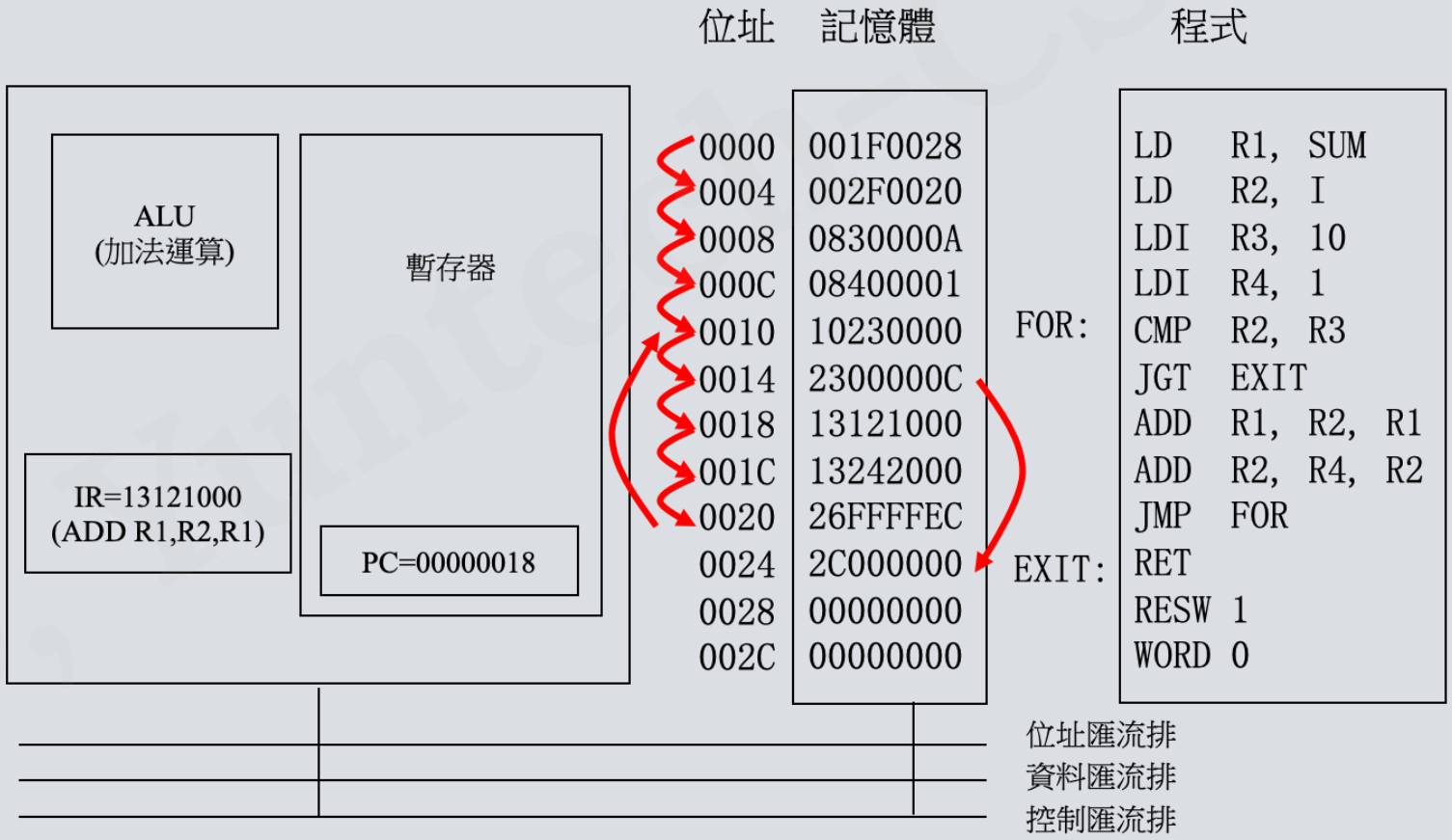
CPU0 的程式執行

- 在整個程式的執行過程中, 指令會一個接著一個從記憶體中依序提取並加以執行, 直到出現跳躍指令為止。

IPSI Lab, Yuntech-CSIE

範例

- 程式從位址 `001F0028` 開始執行，指令為 `LD R1, SUM` 。
- CPU 會依序提取記憶體中的每個指令，直到遇到跳躍指令。



指令提取與執行過程

- CPU 每執行一條指令，程式計數器 (Program Counter, PC) 會增加，以便進入下一條指令。
- 第二個指令位於 `0x0004`，依次提取直到位址 `0x0020`。
- 當遇到跳躍指令時，程序跳至指定的地址繼續執行。
  - 當程式執行到 `JMP FOR` 指令時，程式將跳躍到位址 `0x0010`，繼續執行 `CMP R2, R3` 指令。

IPSI Lab, Yuntech-CSIE

指令的三大執行階段

階段 (a)：提取階段

動作 1、提取指令　　：IR = [PC]

動作 2、更新計數器　　：PC = PC + 4

階段 (b)：解碼階段

動作 3、解碼　　：控制單元對 IR 進行解碼後，設定資料流向開關與 ALU 的運算模式

階段 (c)：執行階段

動作 4、執行　　：資料流入 ALU，經過運算後，流回指定的暫存器

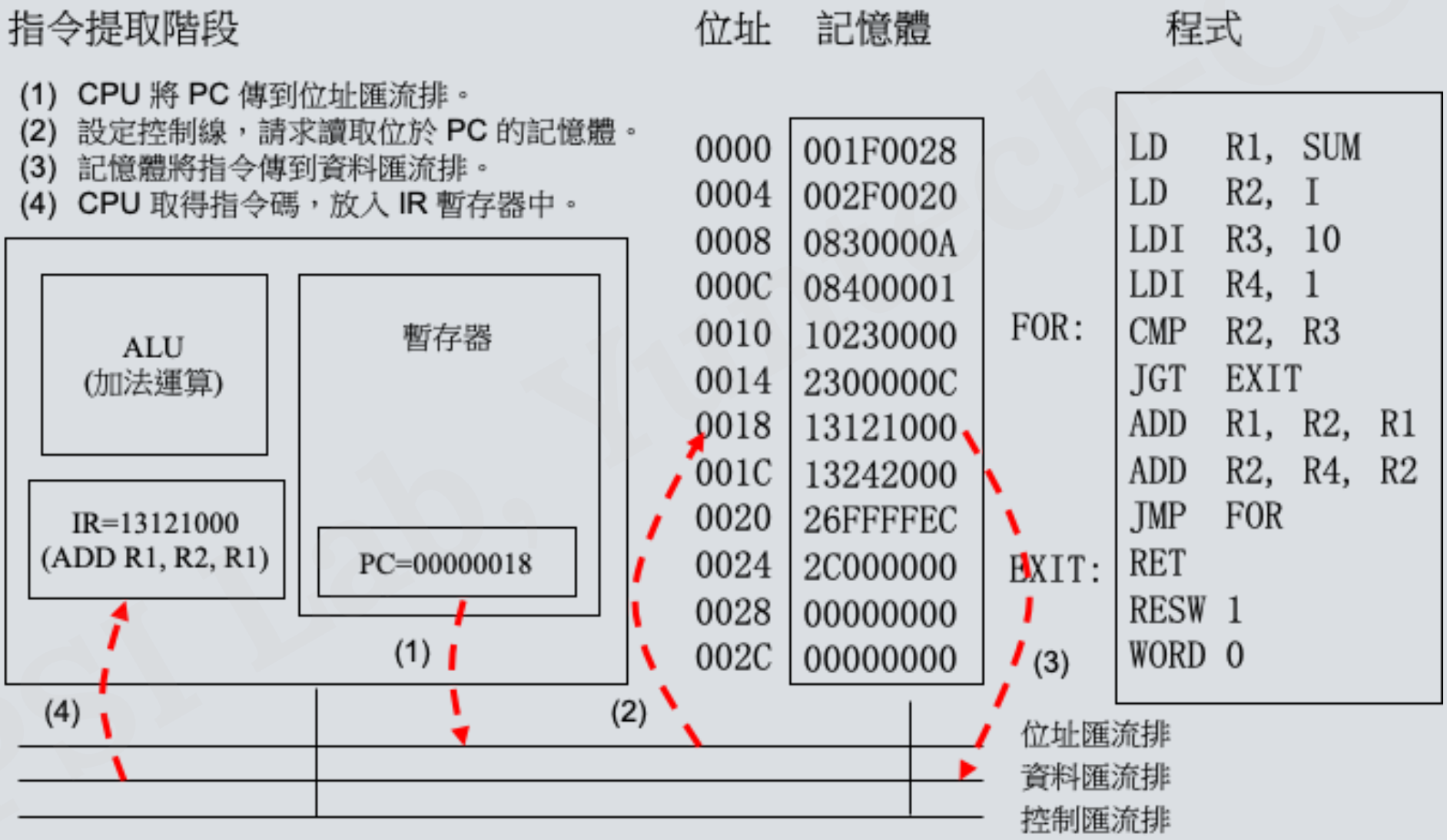
IPSI Lab, Yuntech-CSIE

(a) 提取階段 (Fetch)：

- CPU 會從程式計數器 (PC) 指定的記憶體位址提取指令，並將該指令載入指令暫存器 (IR) 中。
- 程式計數器 (PC) 自動更新，指向下一個要執行的指令位址 (`PC = PC + 4`)。
  註: 通常在大多數 32 位元的處理器架構中，一條指令的長度是 4 個位元組（32 位元）

IPSI Lab, Yuntech-CSIE

指令提取的細部動作


指令提取階段
(1) CPU 將 PC 傳到位址匯流排。
(2) 設定控制線，請求讀取位於 PC 的記憶體。
(3) 記憶體將指令傳到資料匯流排。
(4) CPU 取得指令碼，放入 IR 暫存器中。
ALU
(加法運算)
暫存器
IR=13121000
(ADD R1, R2, R1)
PC=00000018
(1)
(2)
(3)
(4)
位址匯流排
資料匯流排
控制匯流排
位址 記憶體 程式
0000 001F0028
0004 002F0020
0008 0830000A
000C 08400001
0010 10230000
0014 2300000C
0018 13121000
001C 13242000
0020 26FFFFEC
0024 2C000000
0028 00000000
002C 00000000
FOR:
EXIT:
LD R1, SUM
LD R2, I
LDI R3, 10
LDI R4, 1
CMP R2, R3
JGT EXIT
ADD R1, R2, R1
ADD R2, R4, R2
JMP FOR
RET
RESW 1
WORD 0

(b) 解碼階段 (Decode)：

- 控制單元 (Control Unit) 解析指令，確定指令的操作類型、操作數以及 ALU (算術邏輯單元) 的動作。
- 設定資料流方向，控制單元判斷資料是否需要流入 ALU 進行運算。

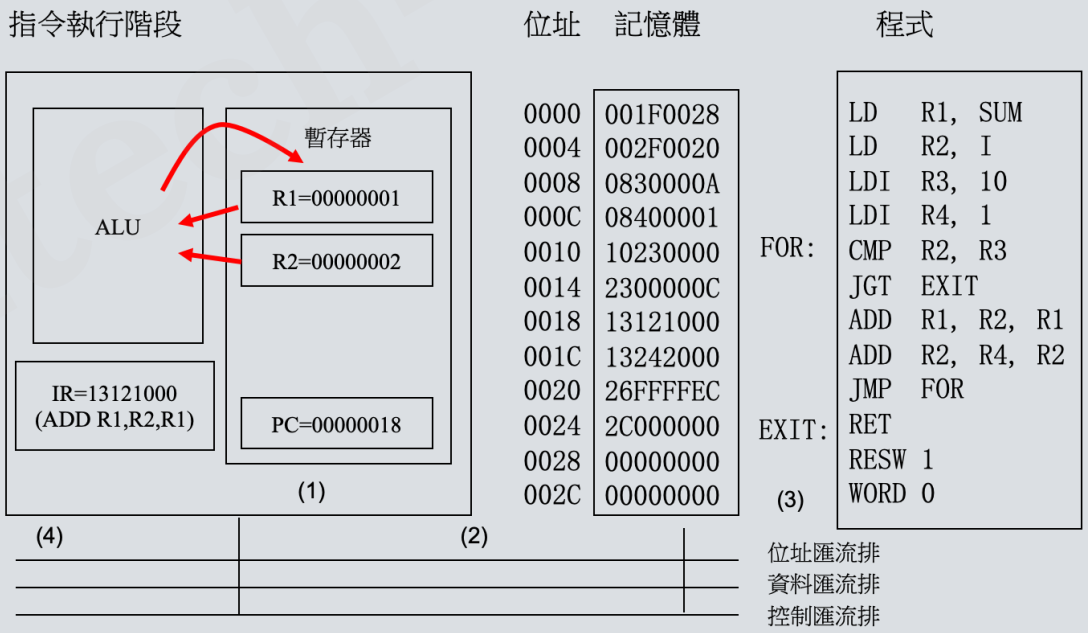
(c) 執行階段 (Execute)：

- 根據解碼結果，ALU 進行具體運算，如加法或邏輯操作。
- 運算完成後，結果被傳送回暫存器，並進行後續操作。

IPSI Lab, Yuntech-CSIE

指令執行範例—加法運算

- **指令：** `ADD R1, R2, R1`
  - 此指令將 R2 和 R1 的數值相加，並將結果存回 R1。
  - `R1` 和 `R2` 的值分別傳送至 ALU，ALU 執行加法運算。



加法運算的過程：

1. **提取資料：**
   - 暫存器 R1 與 R2 的內容被送至 ALU 進行運算。
   - ALU 計算 `R1 + R2` ，並將結果存入暫存器 R1。
2. **結果回寫：**
   - ALU 完成加法運算後，將結果回寫至 R1 暫存器，R1 的值更新為相加後的結果。

IPSI Lab, Yuntech-CSIE

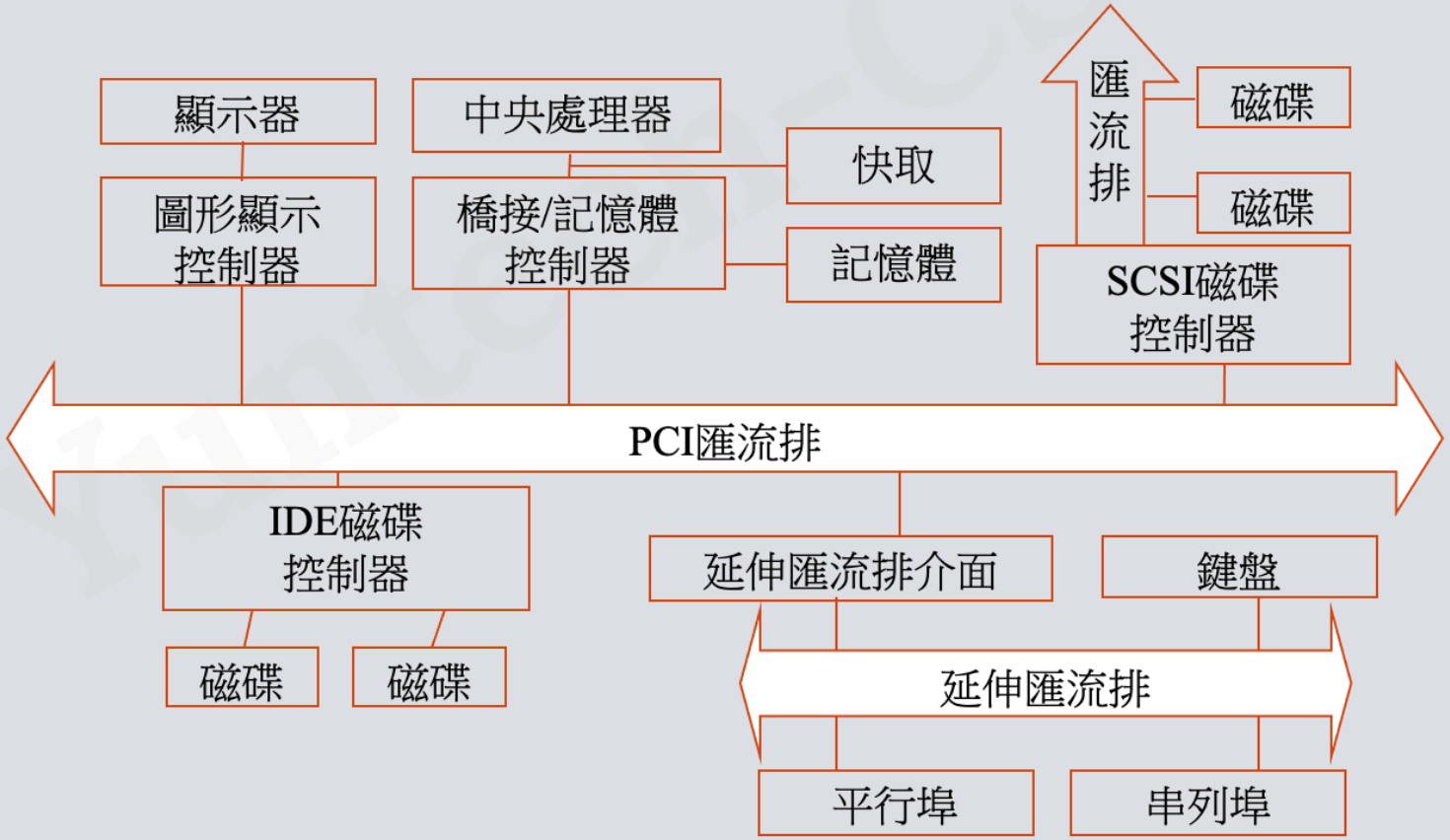
實務案例：IA32 處理器

實務案例：IA32 處理器

- **IA32 處理器背景：**
  - IA32 是 Intel 設計的處理器，屬於 x86 系列家族，是現代 CPU 架構的重要成員之一。
  - 它的架構承襲了過去幾代的設計，歷史悠久，因此其設計特性保留了許多舊的元件和方式。
- **IA32 為什麼複雜？**
  - IA32 處理器的複雜度源自於它需要向後相容，支援舊有的指令集與功能。
  - 這使得它必須同時處理新的 32 位元操作模式以及舊的 16 位元模式，增加了系統複雜性。

IPSI Lab, Yuntech-CSIE

- **IBM PC**
  - IA32 CPU是 IBM PC 的核心，協同匯流排、南北橋晶片、記憶體、顯示裝置等元件，共同形成完整的個人電腦系統。



IA32 的常用暫存器

通用暫存器：EAX
通用暫存器： EBX
通用暫存器： ECX
通用暫存器： EDX

狀態暫存器： EFLAGS

程式計數器：EIP

基底暫存器：EBP
堆疊暫存器：ESP
來源指標：ESI
目的指標：EDI

程式段：CS
堆疊段： SS
資料段： DS
延伸段： ES
延伸段： FS
延伸段： GS

通用暫存器：

- **EAX**：延伸累加器 (Extended Accumulator)，用於算術運算和邏輯運算，常作為加法、乘法等指令的結果暫存器。
- **EBX**：基底暫存器 (Base)，通常用於基底位址的計算，特別是在資料結構和記憶體訪問中。
- **ECX**：計數器暫存器 (Counter)，常用於迴圈和計數操作，如在 `LOOP` 指令中控制迴圈次數。
- **EDX**：資料暫存器 (Data)，用於大數值運算，常與 EAX 搭配使用，特別是處理乘法和除法的餘數運算。

基底與堆疊暫存器：

- **EBP**：基底指標暫存器 (Extended Base Pointer)，主要用於堆疊框架 (Stack Frame) 的基底指標，特別是在函數呼叫時追蹤參數和局部變數的位置。
- **ESP**：堆疊指標暫存器 (Extended Stack Pointer)，指向堆疊的頂端，用於管理函數呼叫過程中的參數和返回地址。
    - 相當於 CPU0 中的 Stack Ponter （SP）

IPSI Lab, Yuntech-CSIE

來源與目的指標：

- **ESI**：來源指標 (Extended Source Index)，常用於指向需要從記憶體讀取的資料來源。
- **EDI**：目的指標 (Extended Destination Index)，常用於指向需要寫入的資料目的地，特別是在字串操作中。

IPSI Lab, Yuntech-CSIE

IA32 的狀態暫存器

- **狀態暫存器 (Extended Flags Register; EFLAGS)**：
  - 進位旗標 (Carry Flag)：指示運算中是否發生進位或借位。
  - 溢位旗標 (Overflow Flag)：反映算術運算結果是否超出範圍。
  - 符號旗標 (Sign Flag)：運算結果的符號，若結果為負數則設定該旗標。
  - 零旗標 (Zero Flag)：當運算結果為零時，設定該旗標。
  - 其他：輔助進位旗標 (Auxiliary Carry Flag)、同位旗標(Parity Flag)等。
- 相當於 CPU0 中的SW 暫存器（R12）

IPSI Lab, Yuntech-CSIE

IA32 的指令計數器

- **指令計數器 (EIP)**：
  - **EIP**：延伸指令指標 (Extended Instruction Pointer)，指向下一個將要執行的指令位址，控制程式的執行流程。EIP 會隨著指令的執行自動更新，或在跳躍指令中被手動設定。
- 相當於 CPU0 中的 Program Counter (PC, R15)

IPSI Lab, Yuntech-CSIE

IA32 的段暫存器

- **CS (Code Segment)**：代碼段，指向正在執行的程式代碼所在的記憶體段。
- **SS (Stack Segment)**：堆疊段，指向堆疊所在的記憶體段，與 ESP 搭配使用。
- **DS (Data Segment)**：資料段，存放全局變數與資料結構。
- **ES (Extended Segment)**：延伸段，用於額外的資料存取。
- **FS、GS**：延伸段，用於多線程環境中的額外段操作，特別是針對現代操作系統中的執行緒管理。

IPSI Lab, Yuntech-CSIE

80286 處理器的區段位址模式

80286 的位元數與暫存器設計：

- **16 位元架構**：在 80286 的時代，x86 系列處理器是 16 位元的。因此，AX, BX, CX, DX 等暫存器都只能處理 16 位元的數值。
- **記憶體限制**：這些暫存器最多只能尋址 64K 的記憶體，這對於當時的應用來說，無法滿足更大的記憶體需求。

段位址與位移的結合：

- **區段+位移模式**：為了突破 64K 記憶體限制，80286 採用了「區段 + 位移」的計算方式，允許最多存取 1MB 的記憶體。
- **公式：** 實際位址 = 區段位址 × 16 + 位移
- 段位址和位移的組合方式使得 16 位元系統能夠運行更大的程式並處理更多的資料。

80386 的 32 位元擴展與兼容性

- **32位元處理器**：在 80386 時代，處理器演變為 32 位元，暫存器可以處理更大數值，並能夠存取更多的記憶體。
- **向後相容性：** 80386 為了與 80286 相容，保留了 AX, BX, CX, DX 等原有的16位元暫存器的名稱。
- **擴展暫存器**：在原來的基礎上擴展為 EAX, EBX, ECX, EDX 等32位元暫存器版本。
  - 這些 32 位元暫存器的較低16 位元中，被劃分為「高位元組 (High Byte)」與「低位元組 (Low Byte)」，例如 AX 包含 AH 和 AL 兩個 8 位元部分。

IPSI Lab, Yuntech-CSIE

EAX 暫存器為例

- **EAX 暫存器的分段：**
  - EAX 是一個 32 位元暫存器，劃分為 AH (8 位元)、AL (8 位元) 以及 AX (16 位元)。這種分段設計允許在不同的位元層級上進行運算。
  - **AH 和 AL 的用途：** 當處理器需要進行 8 位元運算時，AH 和 AL 可以分別用來處理較小的數值，而當進行 16 位元運算時，AX 可以同時參與。



IPSI Lab, Yuntech-CSIE

浮點運算與多媒體擴展 (MMX 和 SIMD)

- **浮點運算的整合：**
  - **浮點運算單元 (FPU)**：在 80386 時代，浮點運算單元仍是外加的硬體晶片，但從 80486 開始，浮點運算功能被整合進 CPU。
  - **浮點暫存器**：CPU 包含了 ST0...ST7 共 8 個浮點暫存器，用來處理浮點數的運算。
- **MMX 和 SIMD 指令集：**
  - **MMX 暫存器**：在 Pentium II 的時代，為了加強多媒體應用，特別引入了 8 個 64 位元的 MMX 暫存器，用於加速影像和聲音處理。
  - **SIMD 指令集**：在 Pentium III 時代，新增了 8 個 XMM 暫存器，支援單指令多資料流 (SIMD) 的平行計算，進一步加快了影像、影片等多媒體的處理速度。

IPSI Lab, Yuntech-CSIE

習題

IPSI Lab, Yuntech-CSIE